© Fernando Barber y Universitat de Valencia. Prohibida la copia o reproducción total o parcial sin previo consentimiento.

Duración: 2h15m **Nombre:**

1. (1.0 ptos) ¿La siguiente afirmación es cierta o falsa? Demuéstralo.

$$\lg\left(\lg\left(2^{\sqrt{n}}\right)\right) \in o\left(\sqrt{n}\right)$$

2. (1.5 ptos)

```
func f(a: vector[1..n] de Natural) dev s: Natural
s ← 0
i ← n
mientras i > 1 ∧ a[i] != n
      para j ← 1 hasta i
            s ← s + a[j]
      fpara
      i ← i - 1
fmientras
```

a) Número de instancias diferentes.

b) Calcula el coste exacto en pasos de los casos mejor y peor.

© Fernando Barber y Universitat de Valencia. Prohibida la copia o reproducción total o parcial sin previo consentimiento.

c) Coste asintótico de la función.

3. (1.0 ptos) Completa la siguiente función recursiva. Ten en cuenta que es posible llamarla con un vector vacío (N = 0).

```
func f(v: Vector[1..N] de Real; i: Natural) dev res: Real

si Cond
    res ← E1
sino
    res ← f(v, E2)
    res ← E3

{Post: res = (Πα: i ≤ α ≤ N: 2*v[α] ) * 3}
```

Cond:
E1:
E2:
E3:

© Fernando Barber y Universitat de Valencia. Prohibida la copia o reproducción total o parcial sin previo consentimiento.

4. (1.25 ptos) Calcula el **coste exacto** en pasos y el **coste asintótico** de la siguiente función recursiva. (Resolver en última página)

```
func f(n: N) dev s: N
si n < 1
      s ← 1
sino
      s ← 1
      s1 ← 2 * f(n-1)
      s2 ← 3 * f(n-1)
      para i ← 1 hasta n-1
            s ← s * s1 - s2
      fpara
fsi
```

5. (1.0 ptos) Implementa el método ***suma(h)*** de la clase ABB. Este método debe sumar todos los datos (TipoDato) del árbol que estén a altura *h*. Se pueden implementar también métodos auxiliares si se considera necesario. La definición de la clase ABB es la siguiente:

```
class ABB {
  public:
    typedef int TipoClave;
    typedef float TipoDato;
    ...
    float suma(unsigned h) const;
  private:
    class NodoBin  {
      public:
        TipoClave clave;
        TipoDato  dato;
        NodoBin * izq;
        NodoBin * der;
            NodoBin(): izq(NULL), der(NULL) {};
            NodoBin(TipoClave c, TipoDato d)
            : clave(c), dato(d), izq(NULL), der(NULL) {};
    };
    typedef NodoBin * PtrNodoBin;
    PtrNodoBin miRaiz;
};
```

© Fernando Barber y Universitat de Valencia. Prohibida la copia o reproducción total o parcial sin previo consentimiento.

6. (1.0 pto) Implementa el método de la clase *Tabla* (tabla hash con encadenamiento) *buscar*, donde cl es la clase del elemento a buscar. El método devolverá *true* si se encuentra la clave y *false* si la clave no existe en la tabla. En dato se devolverá el dato asociado a la clave. Se pueden utilizar otros métodos de la clase Tabla. La clase *Tabla* está declarada de la siguiente manera:

```
class Tabla  {
  public:
    struct ParDato  {
        TipoClave clave;
        TipoDato dato;
    };
    bool buscar(TipoClave cl, TipoDato& dato) const;
    unsigned hash(TipoClave) const;
  private:
    typedef vector<TipoDato> ListaDatos;
    vector<ListaDatos> t;
};
```

© Fernando Barber y Universitat de Valencia. Prohibida la copia o reproducción total o parcial sin previo consentimiento.

7. (1.0 ptos) Dado el siguiente grafo.

	1	2	3	4	5	6	7
1	0	0	4	0	0	1	0
2	2	0	2	0	3	0	0
3	0	0	0	0	0	5	1
4	2	0	1	0	0	0	0
5	0	4	2	0	0	1	0
6	3	1	1	2	0	0	0
7	2	0	0	0	0	1	0

a) Realiza una **búsqueda en profundidad** del grafo empezando en el nodo 1. Dibuja el árbol que representa el recorrido e indica el orden en que se visitarían los nodos.

Árbol:

Nodos:

b) Realiza una **búsqueda en anchura** del grafo empezando en el nodo 1. Dibuja el árbol que representa el recorrido e indica el orden en que se visitarían los nodos.

Árbol:

Nodos:

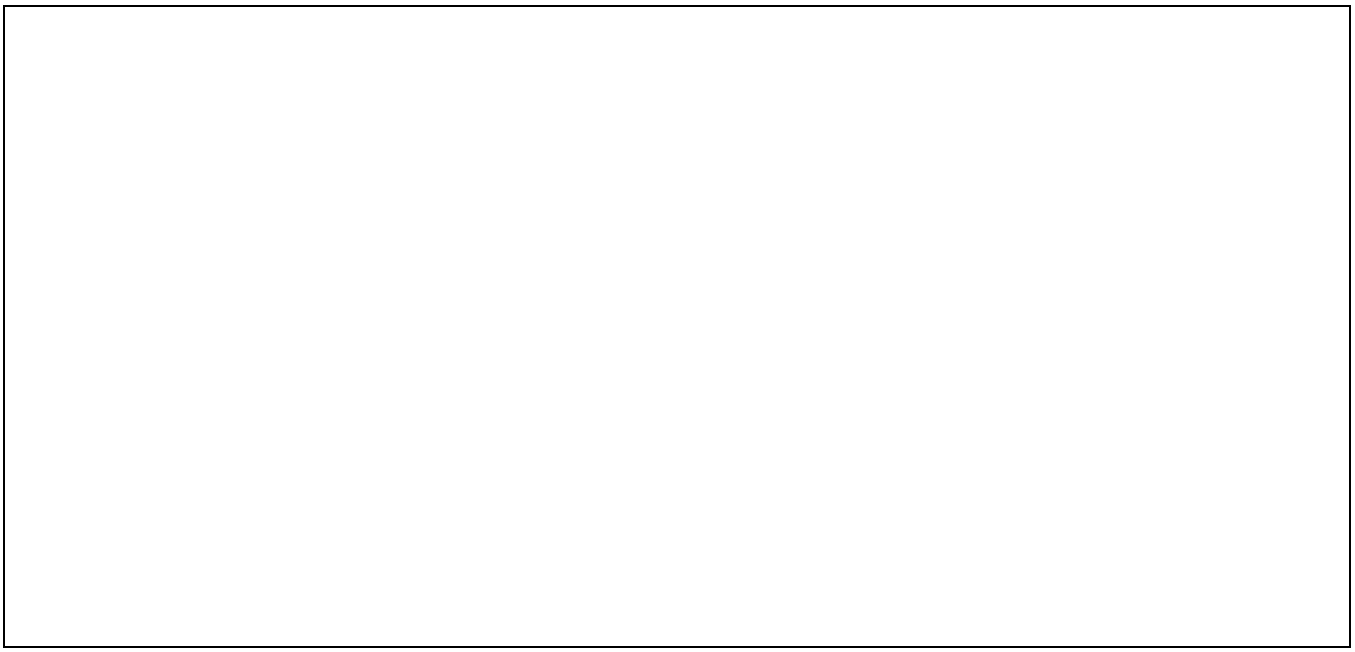
© Fernando Barber y Universitat de Valencia. Prohibida la copia o reproducción total o parcial sin previo consentimiento.

8. (1.0 ptos) Completa el siguiente algoritmo de Prim.

```
ALGORITMO PRIM    // Matriz de adyacencia //
DATOS:      G = (N,A,p) // Grafo no dirigido con pesos
RESULTADO:  vecino: vector[1..n] de 1..n //Mejores arcos factibles
AUX:
            pvecino: vector[1..n] de R    // … y sus pesos.
```

```
pvecino[1] ← -1
Para i ← 2 hasta n hacer
  vecino[i] ← 1
  pvecino[i] ← p[i,1]
fPara
Para i ← 2 hasta n hacer
  min ← inf
  Para j ← 2 hasta n hacer   // Buscamos el arco mínimo
    Si 0 < pvecino[j] < min entonces
      min ← pvecino[j]
      m ← j
    fSi
  fPara
  pvecino[m] ← -1

  Para j ← 2 hasta n hacer //Actualizacion de los arcos
```



```
  fPara
fPara
```

© Fernando Barber y Universitat de Valencia. Prohibida la copia o reproducción total o parcial sin previo consentimiento.

9. (1.25 ptos) Escribe una función en C++ que**,** mediante **backtracking,** muestre por pantalla todas las permutaciones de n elementos con valores {1,2,…,K}, siendo K una constante, que sumen exactamente **x**. Suponer que el operador << funciona con vectores de la STL. Suponer ya definida la función *no_repe*, que comprueba que no hay elementos repetidos desde el inicio del vector hasta la posición i incluida.

```
bool no_repe(vector<unsigned> & d, unsigned i);
```

a)

```
void permRec(unsigned i, vector<unsigned> & d, unsigned x)
{

}
```

b) Escribe la función auxiliar perm que deberá llamar a permRec con los parámetros adecuados.

```
void perm(unsigned x, unsigned n)
{

}
```